

CFD Simulation of Incompressible Flow Past a Bump Using Artificial Compressibility Method

Purna Ananthkrishnan (AE22B003), Yash Shajil (AE22B110)

Department of Aerospace Engineering, Indian Institute of Technology Madras

Abstract: Incompressible Euler equations are solved with an explicit finite volume scheme using the artificial compressibility approach. The scheme is used to simulate the flow past a bump with freestream conditions as $u_\infty = 20$ m/s, $v_\infty = 0$ m/s and gauge pressure, $p_\infty = 0$ Pa. Fluxes at the interfaces are found using an advection based upwinding scheme. An artificial pressure based dissipation (α) is introduced to the mass flux. Computations are performed for two dissipation values $\alpha = 1$ and $\alpha = 0.5$. The larger dissipation is observed to improve stability and accelerated convergence, whereas the lower dissipation requires much larger computational time but is expected to give more accurate results. First-order explicit time integration is used to march towards the steady state solution.

1 Introduction

The incompressible Euler equations are expressed in conservation form (equation 1) to implement into a finite volume scheme

$$M \bar{q}_t + \bar{\nabla} \cdot f = 0 \quad (1)$$

here $\bar{q}$ is

$$\begin{pmatrix} p \\ u \\ v \end{pmatrix}$$

and M is

$$\begin{bmatrix} 1/\beta^2 & 0 & 0 \\ u/\beta^2 & \rho & 0 \\ v/\beta^2 & 0 & \rho \end{bmatrix}$$

Here, β is the artificial compressibility. Upon integration a discretised equation can be obtained as

$$M(\bar{q}_{i,j}^{n+1} - \bar{q}_{i,j}^n) \Delta V + \sum_{k=1}^4 \bar{F}_k \Delta t = 0 \quad (2)$$

The fluxes $\bar{F}$ are constructed using advection upwinding scheme, where the flux at a particular interface is given by

$$\bar{F}_{i+\frac{1}{2}} = \left(m_{i+\frac{1}{2}}^+ \begin{pmatrix} 1 \\ u \\ v \end{pmatrix}_i + m_{i+\frac{1}{2}}^- \begin{pmatrix} 1 \\ u \\ v \end{pmatrix}_{i+1} + p_{i+\frac{1}{2}} \begin{pmatrix} 0 \\ n_x \\ n_y \end{pmatrix}_i \right) A_{i+\frac{1}{2}} \quad (3)$$

with the mass fluxes account for an artificial dissipation (α)

$$\dot{m}_{i+\frac{1}{2}}^+ = \left(\rho U_i^+ + \alpha \frac{p_i - p_{i+1}}{2\lambda_{\max}} \right) A_{i+\frac{1}{2}}, \quad (4)$$

$$\dot{m}_{i+\frac{1}{2}}^- = \left(\rho U_{i+1}^- + \alpha \frac{p_i - p_{i+1}}{2\lambda_{\max}} \right) A_{i+\frac{1}{2}}, \quad (5)$$

where $\lambda_{\max}$ is the maximum wave speed normal to the face and

$$U_i = \vec{v}_i \cdot \hat{n}_{i+\frac{1}{2}}, \quad (6)$$

$$U_{i+1} = \vec{v}_{i+1} \cdot \hat{n}_{i+\frac{1}{2}}, \quad (7)$$

and $\hat{n}_{i+\frac{1}{2}}$ is the unit normal at face $i + \frac{1}{2}$. Here U is the contravariant velocity.

2 Computational Domain

The scheme is used to simulate flow over a bump geometry. A structured curvilinear mesh is used over the domain as given in figure 1, with 97 grid points along the x-direction and 49 grid points along the y-direction.

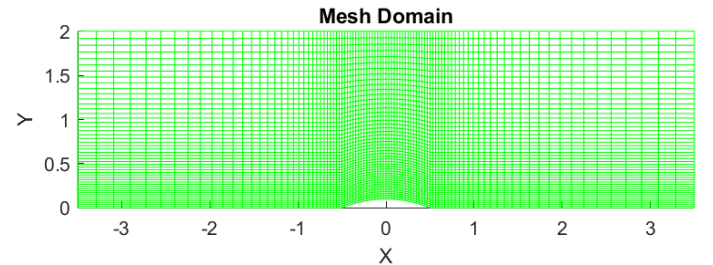


Fig. 1: Meshing of the Computational Domain

3 Numerical Method

3.1 Boundary Conditions

Boundary conditions are imposed at the beginning of every iteration. On the left boundary, Dirichlet boundary conditions are applied such that the u and v velocities of all the ghost cells are u_∞ and v_∞ respectively. Zero pressure gradient is imposed at the left boundary. On the right boundary, zero gradient conditions are applied on u and v velocities, and pressure of the ghost cells along the right boundary are set to p_∞ . The top and bottom boundaries are taken to be slip walls, with zero convection of flux and zero pressure gradient.

3.2 Interface Fluxes

The fluxes at the interface are found using advective upwinding as shown in section 1. The maximum wave speed normal to the face is

found as

$$\lambda_{max} = \frac{|U_{i+\frac{1}{2}}| + \sqrt{U_{i+\frac{1}{2}}^2 + 4\beta^2}}{2} \quad (8)$$

where, $U_{i+\frac{1}{2}} = 0.5 * (U_i + U_{i+1})$. The fluxes through all x-faces ($\bar{F}$) and through all y-faces ($\bar{G}$) are found in the computational domain using this method, for every iteration.

3.3 Residual Calculation

The residual for a given time-step (or a given iteration) is found using the interface fluxes as

$$\bar{R}_{i,j}^n = \bar{F}_i^n - \bar{F}_{i-1}^n + \bar{G}_j^n - \bar{G}_{j-1}^n \quad (9)$$

The residuals obtained are used to update the values of the flow variables at each time step and to check for convergence of the scheme.

3.4 Time Marching

The flow variables (p, u, v) are computed for the subsequent $(n+1)^{th}$ time step using the residuals

$$\bar{q}_{i,j}^{n+1} = \bar{q}_{i,j}^n - \frac{\Delta t_{i,j}}{V_{i,j}} M^{-1} \bar{R}_{i,j}^n \quad (10)$$

The time steps are calculated for each cell individually as

$$\frac{\Delta t_{i,j}}{V_{i,j}} = \frac{CFL}{\sum_{k=1}^4 A_k |\lambda_k|} \quad (11)$$

where,

$$\lambda_k = \frac{|\bar{v}_{i,j} \cdot \hat{n}|_k + \sqrt{(\bar{v}_{i,j} \cdot \hat{n})_k^2 + 4\beta^2}}{2} \quad (12)$$

3.5 Convergence

The residuals are non-dimensionalised by appropriate scales and the norm of this non-dimensional residual is monitored to converge to a specified tolerance. Thus the convergence criterion used is

$$\frac{\left| \begin{bmatrix} \frac{1}{\rho u_\infty} & 0 & 0 \\ 0 & \frac{1}{\rho u_\infty^2} & 0 \\ 0 & 0 & \frac{1}{\rho u_\infty^2} \end{bmatrix} \bar{R}_{i,j}^n \right|}{\left| \begin{bmatrix} \frac{1}{\rho u_\infty} & 0 & 0 \\ 0 & \frac{1}{\rho u_\infty^2} & 0 \\ 0 & 0 & \frac{1}{\rho u_\infty^2} \end{bmatrix} \bar{R}_{i,j}^0 \right|} < \epsilon$$

4 Parameters

4.1 Flow parameters

$$U_\infty = 20 \text{ m/s}$$

$$V_\infty = 0 \text{ m/s}$$

$$P_\infty = 0 \text{ Pa (Gauge)}$$

4.2 Modelling Parameters

- **Tolerance:** The tolerance of the solver is taken to be 10^{-3} .
- **Artificial compressibility:** $\beta = 20 \text{ m/s}$
- **Artificial Dissipation:** Two cases were considered, $\alpha = 0.5$ and $\alpha = 1$.
- **CFL:** For $\alpha = 1$ a CFL value of 0.4 was used in the computation, it provided good stability and fast convergence. However upon reducing α to 0.5 the stability decreased significantly and convergence was achieved for CFL value of 0.08

5 Results

5.1 $\alpha = 0.5$

The convergence behavior was determined by plotting the norm of the residual for every iteration.

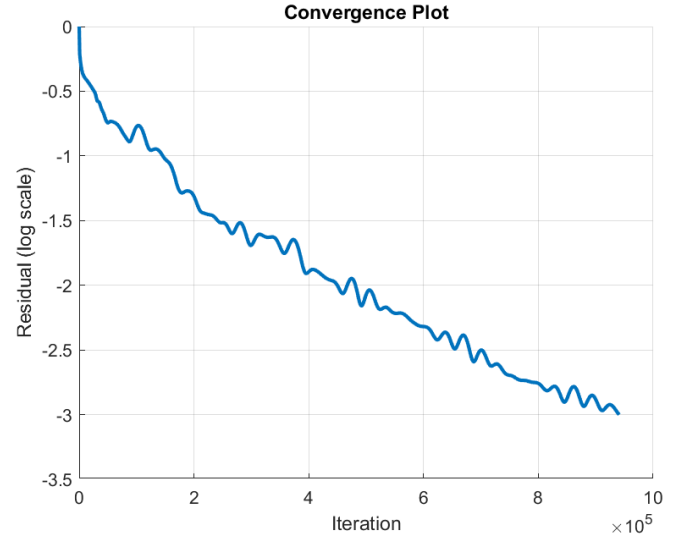


Fig. 2: Convergence Plot, $\alpha = 0.5$

The convergence plot shows convergence with some oscillations, however the number of iterations taken for convergence to occur is very large. It was observed that for higher CFL numbers convergence isn't achieved, and the convergence plots oscillates after reaching a certain number of iterations. The following are the plots depicting the results of the computation

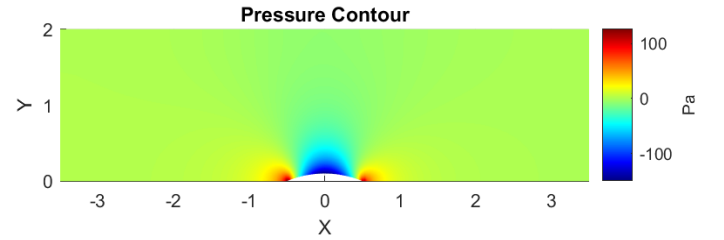


Fig. 3: Pressure Contour, $\alpha = 0.5$

The pressure is expected to be higher at the start and end of the bump (stagnation conditions) and much lower at the top of the bump due to local acceleration. This is captured in the contour plot. Negative values of pressure are achieved as the pressures are all gauge.

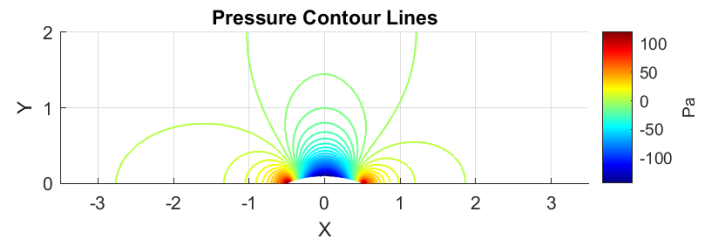


Fig. 4: Constant Pressure Lines, $\alpha = 0.5$

The constant pressure lines are very closely spaced near the bump and grow further apart away from it, signifying that the pressure gradients are large near the bump, as it is expected.

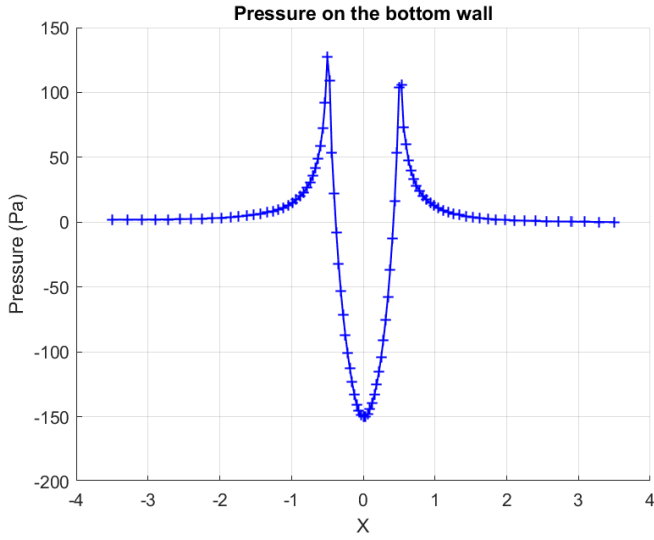


Fig. 5: Pressure Variation along Bottom Wall, $\alpha = 0.5$

The pressure variation along the bottom wall, shows high pressures (stagnation) at the two ends and a drop in pressure at the top of the bump. The pressure at the rear end is observed to be slightly lower than that of the rear end, this could be because of adverse pressure gradients developing on the rear side.

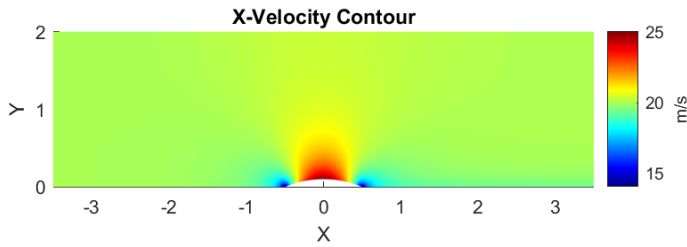


Fig. 6: X-velocity contour, $\alpha = 0.5$

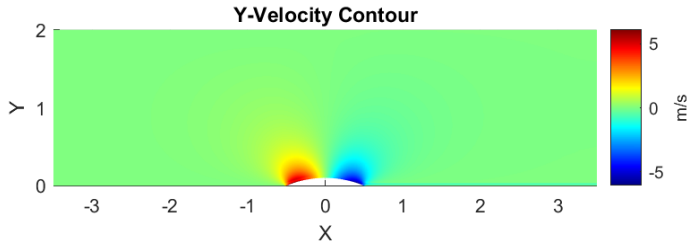


Fig. 7: Y-velocity contour, $\alpha = 0.5$

The velocity contours also show expected behaviour. The x-direction velocity is expected to increase at the top of the bump, where the flow is accelerating along the x-direction. The y-direction velocity is expected to positive in the front half of the bump (upward flow) and negative in the rear part (downward flow) and near zero at the top where the flow is expected to be tangential, hence along x-direction only.

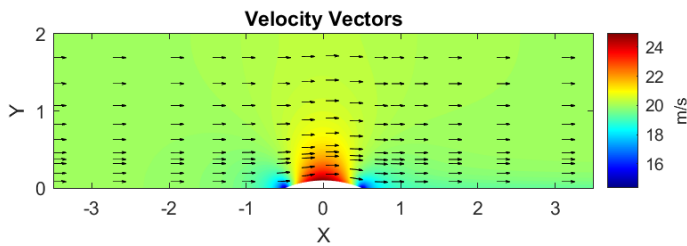


Fig. 8: Streamlines, $\alpha = 0.5$

The velocity vectors are shown with the magnitude as a contour. The velocity is mostly leftward and similar to freestream velocity in the domain, however changes in velocity direction and magnitude are seen near the bump.

5.2 $\alpha = 1$

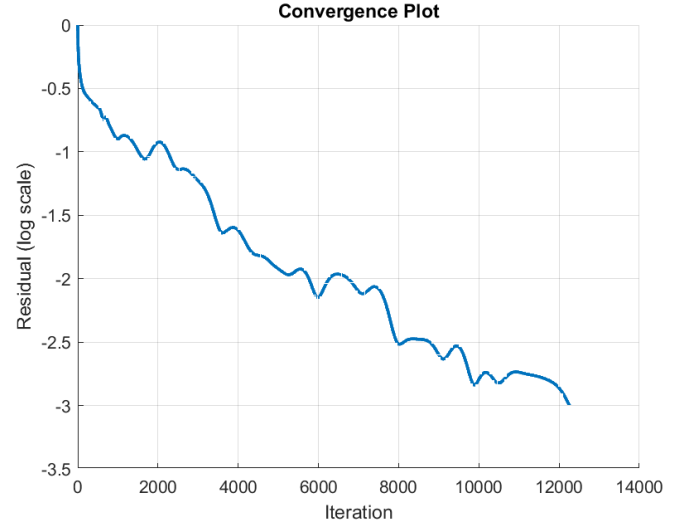


Fig. 9: Convergence Plot, $\alpha = 1$

The convergence for a higher artificial dissipation is seen to be much faster, with convergence at higher CFL values as well. The following are the plots depicting the results of the computation, the trends of the plots obtained are very similar to those obtained for $\alpha = 0.5$ and satisfy the expected physics of flow over such a geometry. The differences in the results obtained for each of the dissipation is elaborated in section 5.3.

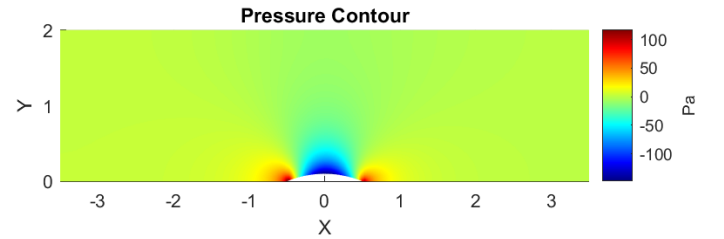


Fig. 10: Pressure Contour, $\alpha = 1$

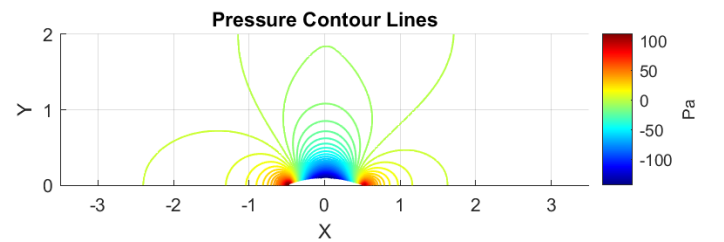


Fig. 11: Constant Pressure Lines, $\alpha = 1$

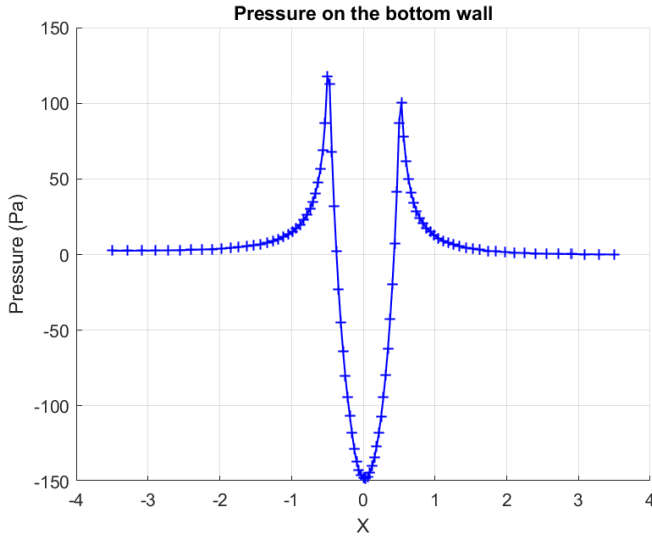


Fig. 12: Pressure Variation along Bottom Wall, $\alpha = 1$

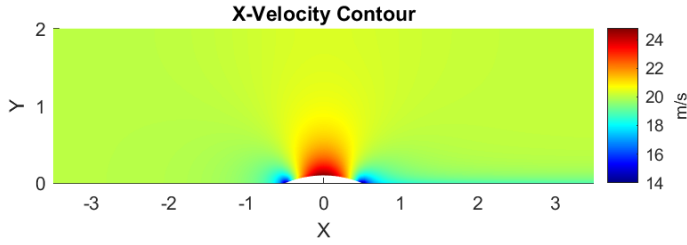


Fig. 13: X-velocity contour, $\alpha = 1$

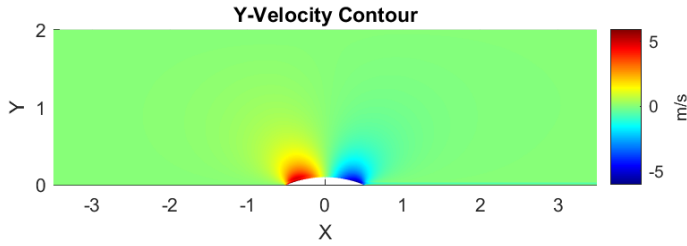


Fig. 14: Y-velocity contour, $\alpha = 1$

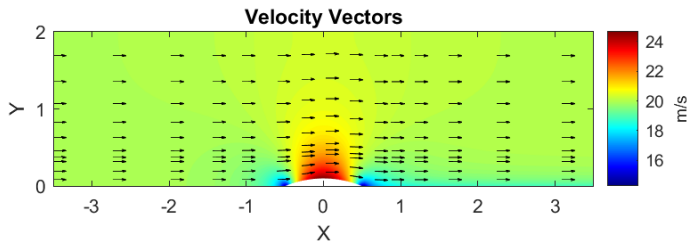


Fig. 15: Streamlines, $\alpha = 1$

6 Conclusion

Incompressible Euler equations were solved to compute the steady state solution for the flow past a bump geometry. The finite volume method was adopted with an explicit time marching scheme that incorporates artificial compressibility and interface flux determination using advective upwinding while also incorporating artificial dissipation to the mass flux.

Computations were performed for two values of artificial dissipation $\alpha = 0.5$ and 1. The higher α gave a much more stable and fast converging solution, whereas the lower dissipation required a very low CFL number to converge and required significantly higher computational time.

The results obtained using the solver closely matched the expected results from the physics of the problem at hand, thus validating the accuracy of the solver.

5.3 Comparison of Results for Different Dissipations

- Higher dissipation values results in more stability, with faster convergence at higher CFL values. Whereas the lower dissipation values results in significantly higher number of iterations to converge and requires a much lower CFL value to be stable.
- A lower dissipation is expected to converge to a more accurate solution as introducing as artificial dissipation into the flow results in damping of the physical gradients thus enhancing convergence and stability but also resulting in a lesser accurate representation of the flow field.

Appendix

Code

```
1  clc; clear; close all;
2
3  %% Flow parameters
4  u_inf = 20;
5  v_inf = 0;
6  p_inf = 0;
7  rho = 1.28;
8  %% Modelling parameters
9  alpha = 1; % Artificial dissipation
10 beta = 20; % Artificial compressibility
11 CFL = 0.5;
12 tol = 1e-3;
13
14 % Reset flag so that if program terminates before converging then we dont
15 % have to run from start again
16 % 1 : Start from 0th iteration and want the node values to be backed up every 100 iteration
17 % 0 : Start from previous terminated point and uses u, v, p from backup_file.txt
18 % -1 : Starts from 0th iteration and no backup created
19 reset = 1;
20
21 %% Reading and storing mesh data
22 data = readtable('bumpgrid.dat.txt');
23 x = data(:,1); y = data(:,2);
24 imax = int32(x(1)); jmax = int32(y(1));
25 x = x(2:end); y = y(2:end);
26 mesh_data_x = zeros(imax,jmax); mesh_data_y = zeros(imax,jmax);
27 line = 1;
28 for j = 1:jmax
29     for i = 1:imax
30         mesh_data_x(i,j) = x(line);
31         mesh_data_y(i,j) = y(line);
32         line = line + 1;
33     end
34 end
35 %% Mesh Visualization
36 fprintf('-----Plotting Mesh-----\n\n')
37
38 figure;
39 hold on;
40
41 plot(mesh_data_x, mesh_data_y, '-g')
42 plot(mesh_data_x', mesh_data_y', '-g')
43 title('Mesh Domain'); xlabel('X'); ylabel('Y');
44 axis equal tight
45
46 fprintf('-----Mesh Plot Done-----\n\n')
47 %% Finding normal vector for faces
48 n_xface_x = zeros(imax,jmax - 1); n_xface_y = zeros(imax,jmax - 1);
49 n_yface_x = zeros(imax-1,jmax); n_yface_y = zeros(imax-1,jmax);
50 A_x = zeros(imax,jmax-1); A_y = zeros(imax-1,jmax);
51 % y face
52 % (i,j) will be area or normal of y-face bw pts (i,j) and (i+1,j)
53 for j = 1 : jmax
54     for i = 1 : imax - 1
55         t_x = mesh_data_x(i,j) - mesh_data_x(i+1,j);
56         t_y = mesh_data_y(i,j) - mesh_data_y(i+1,j);
57         n_yface_x(i,j) = t_y; n_yface_y(i,j) = -t_x;
58         A_y(i,j) = sqrt(t_x^2 + t_y^2);
59     end
60 end
61 % normalisation
62 n_yface_x = n_yface_x./A_y; n_yface_y = n_yface_y./A_y;
63 % x face
64 % (i,j) will be area or normal of x-face bw pts (i,j) and (i,j+1)
65 for j = 1 : jmax - 1
66     for i = 1 : imax
67         t_x = mesh_data_x(i,j+1) - mesh_data_x(i,j);
68         t_y = mesh_data_y(i,j+1) - mesh_data_y(i,j);
69         n_xface_x(i,j) = t_y; n_xface_y(i,j) = -t_x;
```

```

70         A_x(i,j) = sqrt(t_x^2 + t_y^2);
71     end
72 end
73 % normalisation
74 n_xface_x = n_xface_x./A_x; n_xface_y = n_xface_y./A_x;
75 %% Variable Initialization
76 F1 = zeros(imax,jmax-1);
77 F2 = zeros(imax,jmax-1);
78 F3 = zeros(imax,jmax-1);
79 G1 = zeros(imax-1,jmax);
80 G2 = zeros(imax-1,jmax);
81 G3 = zeros(imax-1,jmax);
82
83 if (reset == 1)
84     u = u_inf*ones(imax+1, jmax+1);
85     v = v_inf*ones(imax+1, jmax+1);
86     p = p_inf*ones(imax+1, jmax+1);
87     fid_resnorm = fopen("backup_resnorm.txt",'w');
88     fclose(fid_resnorm);
89
90 elseif (reset == -1)
91     u = u_inf*ones(imax+1, jmax+1);
92     v = v_inf*ones(imax+1, jmax+1);
93     p = p_inf*ones(imax+1, jmax+1);
94
95 elseif (reset == 0)
96     fprintf('-----Reading from backup files-----\n\n')
97
98     prev_data = readtable('backup_file.txt');
99     prev_u = prev_data(:,1); prev_v = prev_data(:,2); prev_p = prev_data(:,3);
100     u = zeros(imax+1, jmax+1);
101     v = zeros(imax+1, jmax+1);
102     p = zeros(imax+1, jmax+1);
103     line = 1;
104     for j = 1:jmax+1
105         for i = 1:imax+1
106             u(i,j) = prev_u(line);
107             v(i,j) = prev_v(line);
108             p(i,j) = prev_p(line);
109             line = line + 1;
110         end
111     end
112
113     prev_data = readtable('backup_resnorm.txt');
114     prev_itn = prev_data(:,1); prev_resnorm = prev_data(:,2);
115 end
116
117 %% Main loop
118 fprintf('-----Entering main loop-----\n\n')
119
120 if ((reset == 1) || (reset == -1))
121     itn = 1;
122 elseif (reset == 0)
123     itn = prev_itn(end);
124 end
125
126 resnorm = [];
127 while true
128     %% Boundary conditions and initial values
129
130     % Inflow - Left
131     u(1,:) = u_inf;
132     v(1,:) = v_inf;
133     p(1,:) = p(2,:);
134
135     % Outflow - Right
136     u(imax+1,:) = u(imax,:);
137     v(imax+1,:) = v(imax,:);
138     p(imax+1,:) = p_inf;
139
140     % Slip wall - Top
141     G1(:,jmax) = 0;
142     p(2:imax,jmax+1) = p(2:imax,jmax);

```

```

143 G2(:,jmax) = 0.5*(p(2:imax,jmax) + p(2:imax,jmax+1)).*n_yface_x(:,jmax) .* A_y(:,jmax);
144 G3(:,jmax) = 0.5*(p(2:imax,jmax) + p(2:imax,jmax+1)).*n_yface_y(:,jmax) .* A_y(:,jmax);
145 vn_upper = u(2:imax,jmax).*n_yface_x(:,jmax) + v(2:imax,jmax).*n_yface_y(:,jmax);
146 u(2:imax,jmax+1) = u(2:imax,jmax) - 2*vn_upper.*n_yface_x(:,jmax);
147 v(2:imax,1) = v(2:imax,2) - 2*vn_upper.*n_yface_y(:,jmax);
148
149 % Slip wall - Bottom
150 G1(:,1) = 0;
151 p(2:imax,1) = p(2:imax,2);
152 G2(:,1) = 0.5*(p(2:imax,2) + p(2:imax,1)).*n_yface_x(:,1) .* A_y(:,1);
153 G3(:,1) = 0.5*(p(2:imax,2) + p(2:imax,1)).*n_yface_y(:,1) .* A_y(:,1);
154 vn_lower = u(2:imax,2).*n_yface_x(:,2) + v(2:imax,2).*n_yface_y(:,2);
155 u(2:imax,1) = u(2:imax,2) - 2*vn_lower.*n_yface_x(:,1);
156 v(2:imax,1) = v(2:imax,2) - 2*vn_lower.*n_yface_y(:,1);
157
158 %% Storing face fluxes
159
160 % x face
161 for j = 1:jmax-1
162     for i = 1:imax
163         u_L = u(i,j+1);
164         u_R = u(i+1,j+1);
165         v_L = v(i,j+1);
166         v_R = v(i+1,j+1);
167         p_L = p(i,j+1);
168         p_R = p(i+1,j+1);
169
170         U_i = u_L*n_xface_x(i,j) + v_L*n_xface_y(i,j);
171         U_i_plus_one = u_R*n_xface_x(i,j) + v_R*n_xface_y(i,j);
172
173         u_avg = (u_L + u_R)/2;
174         v_avg = (v_L + v_R)/2;
175         p_avg = (p_L + p_R)/2;
176
177         U_i_plus_half = 0.5*(U_i+U_i_plus_one);
178         c = max_wave_speed(U_i_plus_half,beta);
179
180
181         rho_cont_i_plus_half_plus = rho*max(0,U_i) + alpha/(2*c) * (p_L - p_R);
182         rho_cont_i_plus_half_minus = rho*min(0,U_i_plus_one) + alpha/(2*c) * (p_L -
183             ↪ p_R);
184
185         F1(i,j) = (rho_cont_i_plus_half_plus*1 + rho_cont_i_plus_half_minus*1 +
186             ↪ p_avg*0).*A_x(i,j);
187         F2(i,j) = (rho_cont_i_plus_half_plus*u_L + rho_cont_i_plus_half_minus*u_R +
188             ↪ p_avg*n_xface_x(i,j)).*A_x(i,j);
189         F3(i,j) = (rho_cont_i_plus_half_plus*v_L + rho_cont_i_plus_half_minus*v_R +
190             ↪ p_avg*n_xface_y(i,j)).*A_x(i,j);
191     end
192 end
193
194 % y face
195 for i = 1:imax-1
196     for j = 2:jmax-1
197         u_B = u(i+1,j);
198         u_T = u(i+1,j+1);
199         v_B = v(i+1,j);
200         v_T = v(i+1,j+1);
201         p_B = p(i+1,j);
202         p_T = p(i+1,j+1);
203
204         U_i = u_B*n_yface_x(i,j) + v_B*n_yface_y(i,j);
205         U_i_plus_one = u_T*n_yface_x(i,j) + v_T*n_yface_y(i,j);
206
207         u_avg = (u_B + u_T)/2;
208         v_avg = (v_B + v_T)/2;
209         p_avg = (p_B + p_T)/2;
210
211         U_i_plus_half = 0.5*(U_i+U_i_plus_one);
212         c = max_wave_speed(U_i_plus_half,beta);
213
214         rho_cont_i_plus_half_plus = rho*max(0,U_i) + alpha/(2*c) * (p_B - p_T);

```

```

211     rho_cont_i_plus_half_minus = rho*min(0,U_i_plus_one) + alpha/(2*c) * (p_B -
212     ↪ p_T);
213
214     G1(i,j) = (rho_cont_i_plus_half_plus*1 + rho_cont_i_plus_half_minus*1 +
215     ↪ p_avg*0).*A_y(i,j);
216     G2(i,j) = (rho_cont_i_plus_half_plus*u_B + rho_cont_i_plus_half_minus*u_T +
217     ↪ p_avg*n_yface_x(i,j)).*A_y(i,j);
218     G3(i,j) = (rho_cont_i_plus_half_plus*v_B + rho_cont_i_plus_half_minus*v_T +
219     ↪ p_avg*n_yface_y(i,j)).*A_y(i,j);
220
221     end
222 end
223
224 %% Residual Calculation
225
226 Res1 = zeros(imax-1, jmax-1);
227 Res2 = zeros(imax-1, jmax-1);
228 Res3 = zeros(imax-1, jmax-1);
229
230 for i = 2:imax
231     for j = 2:jmax
232         Res1(i-1,j-1) = F1(i,j-1) - F1(i-1,j-1) + G1(i-1,j) - G1(i-1,j-1);
233         Res2(i-1,j-1) = F2(i,j-1) - F2(i-1,j-1) + G2(i-1,j) - G2(i-1,j-1);
234         Res3(i-1,j-1) = F3(i,j-1) - F3(i-1,j-1) + G3(i-1,j) - G3(i-1,j-1);
235     end
236 end
237
238 %% Time Step Estimation
239
240 dt_by_Vol = zeros(imax-1,jmax-1);
241
242 for j = 2:jmax
243     for i = 2:imax
244         Vn1 = u(i,j) * n_xface_x(i,j-1) + v(i,j) * n_xface_y(i,j-1);
245         Vn2 = u(i,j) * n_yface_x(i-1,j) + v(i,j) * n_yface_y(i-1,j);
246         Vn3 = u(i,j) * n_xface_x(i-1,j-1) + v(i,j) * n_xface_y(i-1,j-1);
247         Vn4 = u(i,j) * n_yface_x(i-1,j-1) + v(i,j) * n_yface_y(i-1,j-1);
248         lambda1 = max_wave_speed(Vn1, beta);
249         lambda2 = max_wave_speed(Vn2, beta);
250         lambda3 = max_wave_speed(Vn3, beta);
251         lambda4 = max_wave_speed(Vn4, beta);
252         temp = A_x(i,j-1)*lambda1 + A_y(i-1,j)*lambda2 + A_x(i-1,j-1)*lambda3...
253         + A_y(i-1,j-1)*lambda4;
254         dt_by_Vol(i-1,j-1) = CFL/temp;
255     end
256 end
257
258 %% Update loop
259 u_old = u; p_old = p; v_old = v;
260 f = 1;
261 for j = 2:jmax
262     for i = 2:imax
263         u_new = u(i,j) - (dt_by_Vol(i-1,j-1)/rho) * (Res2(i-1,j-1) -
264         ↪ u(i,j)*Res1(i-1,j-1));
265         v_new = v(i,j) - (dt_by_Vol(i-1,j-1)/rho) * (Res3(i-1,j-1) -
266         ↪ v(i,j)*Res1(i-1,j-1));
267         p_new = p(i,j) - (beta^2*dt_by_Vol(i-1,j-1)/rho) * Res1(i-1,j-1);
268         u(i,j) = f*u_new + (1-f)*u(i,j);
269         v(i,j) = f*v_new + (1-f)*v(i,j);
270         p(i,j) = f*p_new + (1-f)*p(i,j);
271     end
272 end
273
274 %% Residual Norm calculations
275
276 resnorm(itn) = 0;
277
278 scale1 = rho*u_inf;
279 scale2 = rho*u_inf*u_inf;
280 scale3 = rho*u_inf*u_inf;
281
282 for j = 2:jmax
283     for i = 2:imax

```

```

278         resnorm(itn) = resnorm(itn) + (Res1(i-1,j-1)/scale1)^2 + (Res2(i-1,j-1)/scale2)^2
279         ↪ + (Res3(i-1,j-1)/scale3)^2;
280     end
281     resnorm(itn) = sqrt(resnorm(itn));
282
283     if (reset == 1)
284         if itn == 1
285             resnorm_0 = resnorm(itn);
286             fid_resnorm = fopen('backup_resnorm.txt','a');
287             fprintf(fid_resnorm, '%f %f\n', 0, resnorm_0);
288             fclose(fid_resnorm);
289         end
290     elseif (reset == 0)
291         if itn == prev_itn(end)
292             prev_data = readtable('backup_resnorm.txt');
293             resnorm_0 = prev_data{1,2};
294         end
295     end
296
297     if (resnorm(itn)/resnorm_0) < tol
298         break
299     end
300
301     if (mod(itn,10) == 0)
302         fprintf('Iteration : %d | Resnorm Ratio : %f\n', itn, resnorm(itn)/resnorm_0)
303         % fprintf('Max delta p : %f | Max delta u : %f | Max delta v : %f \n\n', ...
304         %         max(abs(p(:) - p_old(:))), ...
305         %         max(abs(u(:) - u_old(:))), ...
306         %         max(abs(v(:) - v_old(:))) );
307     % end
308
309     if ((mod(itn,100) == 0) && (reset ~= -1))
310         % Every 100 iterations, x velocity, y velocity and pressure will be
311         % stored in 'backup_file.txt'. In case of program terminating before
312         % convergence we can use this backup file to start off from n*100 iteration
313         % by setting reset variable as 0, rather from scratch again.
314         fid_backup = fopen("backup_file.txt","w");
315         fprintf('-----Writing the backup file-----\n\n')
316         for j = 1:jmax+1
317             for i = 1:imax+1
318                 fprintf(fid_backup, '%f %f %f\n', u(i,j), v(i,j), p(i,j));
319             end
320         end
321         fclose(fid_backup);
322
323         fid_resnorm = fopen('backup_resnorm.txt','a');
324         fprintf(fid_resnorm, '%d %f\n', itn, resnorm(itn));
325         fclose(fid_resnorm);
326     end
327
328     itn = itn + 1;
329 end
330 fprintf('-----Solution Converged-----\n\n')
331
332 figure; hold on;
333 plot(1:itn, log10(resnorm/resnorm_0), '-');
334 xlabel('Iteration');
335 ylabel('Residual (log scale)');
336 grid on;
337 %% Boundary Conditions
338 % Inflow - Left
339 u(1,:) = u_inf;
340 v(1,:) = v_inf;
341 p(1,:) = p(2,:);
342
343 % Outflow - Right
344 u(imax+1,:) = u(imax,:);
345 v(imax+1,:) = v(imax,:);
346 p(imax+1,:) = p_inf;
347
348 % Slip wall - Top
349 G1(:,jmax) = 0;

```

```

350 p(2:imax,jmax+1) = p(2:imax,jmax);
351 G2(:,jmax) = 0.5*(p(2:imax,jmax) + p(2:imax,jmax+1)).*n_yface_x(:,jmax) .* A_y(:,jmax);
352 G3(:,jmax) = 0.5*(p(2:imax,jmax) + p(2:imax,jmax+1)).*n_yface_y(:,jmax) .* A_y(:,jmax);
353 vn_upper = u(2:imax,jmax).*n_yface_x(:,jmax) + v(2:imax,jmax).*n_yface_y(:,jmax);
354 u(2:imax,jmax+1) = u(2:imax,jmax) - 2*vn_upper.*n_yface_x(:,jmax);
355 v(2:imax,1) = v(2:imax,2) - 2*vn_upper.*n_yface_y(:,jmax);
356
357 % Slip wall - Bottom
358 G1(:,1) = 0;
359 p(2:imax,1) = p(2:imax,2);
360 G2(:,1) = 0.5*(p(2:imax,2) + p(2:imax,1)).*n_yface_x(:,1) .* A_y(:,1);
361 G3(:,1) = 0.5*(p(2:imax,2) + p(2:imax,1)).*n_yface_y(:,1) .* A_y(:,1);
362 vn_lower = u(2:imax,2).*n_yface_x(:,2) + v(2:imax,2).*n_yface_y(:,2);
363 u(2:imax,1) = u(2:imax,2) - 2*vn_lower.*n_yface_x(:,1);
364 v(2:imax,1) = v(2:imax,2) - 2*vn_lower.*n_yface_y(:,1);
365
366 %% Writing Output to File
367 % Used for restarting
368 %% Post Processing
369
370 fprintf('-----Post Processing-----\n\n')
371 str1 = datetime('now', 'Format', 'yyyy_MM_dd_HH_mm_ss');
372 str2 = sprintf('%f_%f',alpha(1), beta, CFL);
373
374 % File name format 'year_month_day_hour_min_sec_alpha_beta_CFL.txt'
375 output_file_name = string(str1) + str2 + '.txt';
376 fid_output = fopen(output_file_name,'w');
377
378 for j = 1:jmax
379     for i = 1:imax
380         u_node = 0.25*(u(i,j) + u(i+1,j) + u(i+1,j+1) + u(i,j+1));
381         v_node = 0.25*(v(i,j) + v(i+1,j) + v(i+1,j+1) + v(i,j+1));
382         p_node = 0.25*(p(i,j) + p(i+1,j) + p(i+1,j+1) + p(i,j+1));
383         fprintf(fid_output,'%f %f %f\n', u_node, v_node, p_node);
384     end
385 end
386
387 fclose(fid_output);
388
389 %%
390 u_node = zeros(imax,jmax);
391 v_node = zeros(imax,jmax);
392 p_node = zeros(imax,jmax);
393
394 % output_file_name = '2025_09_28_18_20_09_0.500000_20.000000_0.008000.txt';
395
396 output_data = readtable(output_file_name);
397 output_u = output_data{:,1}; output_v = output_data{:,2}; output_p = output_data{:,3};
398
399 line = 1;
400 for j = 1:jmax
401     for i = 1:imax
402         u_node(i,j) = output_u(line);
403         v_node(i,j) = output_v(line);
404         p_node(i,j) = output_p(line);
405         line = line + 1;
406     end
407 end
408
409
410 fprintf('-----Plotting Results-----\n\n')
411
412 % X - Velocity Contour
413 figure; hold on
414 surf(mesh_data_x, mesh_data_y, u_node);
415 a = colorbar;
416 ylabel(a, 'm/s')
417 colormap('jet')
418 shading interp
419 title('X-Velocity Contour'); xlabel('X'); ylabel('Y')
420 axis equal tight
421
422 % Y - Velocity Contour

```

```

423 figure; hold on
424 surf(mesh_data_x, mesh_data_y, v_node);
425 a = colorbar;
426 ylabel(a, 'm/s')
427 colormap('jet')
428 shading interp
429 title('Y-Velocity Contour'); xlabel('X'); ylabel('Y')
430 grid('on')
431 axis equal tight
432
433 % Pressure Contour
434 figure; hold on
435 surf(mesh_data_x, mesh_data_y, p_node);
436 a = colorbar;
437 ylabel(a, 'Pa')
438 colormap('jet')
439 shading interp
440 title('Pressure Contour'); xlabel('X'); ylabel('Y')
441 grid('on')
442 axis equal tight
443
444 % Pressure Contour Lines
445 figure; hold on
446 contour(mesh_data_x, mesh_data_y, p_node, 50, 'LineWidth', 1);
447 a = colorbar;
448 ylabel(a, 'Pa')
449 colormap('jet')
450 shading interp
451 title('Pressure Contour Lines'); xlabel('X'); ylabel('Y')
452 grid('on')
453 axis equal tight
454
455 % Pressure on the bottom wall
456 figure; hold on
457 plot(mesh_data_x(:,1), p_node(:,1), '-+b', 'LineWidth', 1);
458 title('Pressure on the bottom wall'); xlabel('X'); ylabel('Pressure (Pa)')
459 grid('on')
460
461 % Velocity vector
462 figure;
463 ylines = [5, 10, 17, 15, 20, 25, 30, 35, 40, 45]; xlines = [1, 5, 10, 15, 20, 30, 40, 50, 60,
↳ 70, 75, 80, 85 90, 95];
464 contourf(mesh_data_x, mesh_data_y, sqrt(u_node.^2 + v_node.^2), 100, 'LineStyle', 'none'); hold
↳ on
465 quiver(mesh_data_x(xlines, ylines), mesh_data_y(xlines, ylines), u_node(xlines,
↳ ylines)./sqrt(u_node(xlines, ylines).^2 + v_node(xlines, ylines).^2), v_node(xlines,
↳ ylines)./sqrt(u_node(xlines, ylines).^2 + v_node(xlines, ylines).^2), 0.25,
↳ 'k', 'ShowArrowHead', 'on')
466 a = colorbar;
467 ylabel(a, 'm/s')
468 colormap('jet')
469 shading interp
470 title('Velocity Vectors'); xlabel('X'); ylabel('Y')
471 axis equal tight
472
473
474 %% Functions
475 function lambda = max_wave_speed(U, beta)
476 lambda = (abs(U) + sqrt(U*U + 4*beta^2))/2;
477 end

```